ASTRAIOS

Technical Whitepaper

Quantitative ML Trading Platform · v2.0 · May 2026

Astraios is a full-stack quantitative trading platform for cryptocurrency derivatives. It combines a CNN-Transformer signal engine trained on 27 USDT perpetual futures, real-time Bybit mainnet and demo execution, an autonomous auto-trader with per-user risk controls, and a React/FastAPI dashboard with 1-second live position updates. The v5 model achieves 55.3% directional accuracy with a Sharpe ratio of +5.52 and +0.65% expected value per trade at 2:1 risk-reward — validated on a 41,634-sample out-of-sample test set.

Contents

1. System Architecture

Astraios follows a monorepo structure: a Python FastAPI backend, a React SPA, PostgreSQL persistence, and a decoupled ML pipeline for AWS SageMaker training. All services run from a single uvicorn process on a Linux EC2 instance. Bybit calls are proxied through a US-exit HTTP proxy to bypass regional API blocks.

Layer	Technology	Role
API Server	FastAPI + Uvicorn	REST endpoints, SPA serving, lifespan hooks
Database	PostgreSQL + asyncpg	Users, signals, positions, auto-trade logs
Background Jobs	asyncio tasks	Signal refresh 15 min · price refresh 5 min · auto-trader
Market Data	Bybit v5 API + Binance Futures	Klines, tickers, OI, funding, L/S ratio
ML Engine	PyTorch + AWS SageMaker	CNN-Transformer training and inference
Frontend	React 19 + Vite 6	SPA: dashboard, charts, trade terminal, auto-trade
Charts	lightweight-charts v4	Candlestick + volume, 1 s poll
Auth	JWT (python-jose) + bcrypt	Per-user, 24h expiry
API Key Crypto	Fernet symmetric encryption	Bybit keys encrypted at rest in PostgreSQL
Rate Limiting	slowapi	5/min login · 10/min register · 10/min orders
Migrations	Alembic	Schema versioning
Proxy	HTTP/HTTPS forward proxy	Routes Bybit/Binance calls from US IP

1.1 Data Flow

On startup the lifespan hook creates all tables and launches three background tasks. Signal generation runs every 15 minutes per symbol: fetch 200 1h candles from Bybit, engineer 28 features, run the MarketTransformer, apply the confidence gate ($|p_{buy} - p_{sell}| \geq 0.10$), write one signal per ticker per user to PostgreSQL, then invoke the auto-trader for all enabled users. Price refresh runs every 5 minutes for open paper positions. The frontend polls Bybit wallet and positions every 1 second via separate browser fetch loops.

2. MarketTransformer v5 — Signal Engine

The signal engine is a CNN-Transformer architecture. A parallel 1D CNN front-end extracts local candlestick patterns (3-, 5-, and 7-bar windows) before the Transformer encoder attends to 48 bars of context. The model outputs a binary classification: BUY (label 1) or SELL (label 0), trained on return-quantile labels derived from 24-bar forward returns.

Component	Configuration
CNN front-end	3 parallel Conv1d (kernel 3/5/7, d_model//3 channels each) → GELU → concat
d_model	66 (22 channels × 3 CNN paths)

Positional encoding	Sinusoidal, max_len=48
Encoder layers	3 × TransformerEncoderLayer (pre-norm)
Attention heads	2
Feed-forward dim	256
Activation	GELU
Dropout	0.15 (training) / 0.0 (inference)
Classification head	Linear(66→256) → GELU → Dropout → Linear(256→2)
Output	2 logits → softmax → BUY/SELL + confidence
Sequence length	48 bars (48 hours of 1h candles)
Input features	28
Total parameters	~85K
Model size	328 KB (model.pt)

2.1 Why CNN + Transformer

A pure Transformer struggles to learn short-range candlestick structure (engulfing patterns, hammer/doji, 3-bar momentum bursts) because self-attention is permutation-equivariant — it has no inductive bias for local ordering. The parallel CNN front-end gives the model explicit local receptive fields at 3h, 5h, and 7h windows before the Transformer encoder captures long-range dependencies across the full 48-bar sequence.

3. Feature Engineering — 28 Features

All features are computed per-symbol from raw OHLCV kline data collected from Binance Futures (funding rates, klines) and Bybit (open interest, long/short). A StandardScaler fitted on training data (no val leakage) normalises inputs; scaler parameters are saved in config.json for consistent inference. Values are clipped to ±5σ before entering the model.

Feature	Description	Group
returns	1-bar pct_change(close)	Price
ema_ratio_8	close / EMA(close, 8)	Trend
ema_ratio_21	close / EMA(close, 21)	Trend
ema_ratio_50	close / EMA(close, 50)	Trend
close_position	(close – low) / (high – low)	Price
rsi_14	RSI(14)	Momentum
macd_hist	MACD(12,26) – Signal(9)	Momentum
bb_pct	(close – BB_lower) / (BB_upper – BB_lower)	Momentum
mom_20	close / close[–20] – 1	Momentum
atr_norm	ATR(14) / close	Volatility

rolling_vol_5	std(returns, 5)	Volatility
rolling_vol_20	std(returns, 20)	Volatility
vol_ratio	volume / MA(volume, 20)	Volume
taker_buy_ratio	taker_buy_vol / total_vol	Volume
taker_buy_pressure	(taker_buy_ratio – 0.5) × vol_ratio	Volume
ret_lag_1	returns shifted 1 bar	Lag
ret_lag_3	returns shifted 3 bars	Lag
funding_rate	Binance 8h funding rate (ffilled to 1h)	Microstructure
funding_rate_ma8	8-bar rolling mean of funding_rate	Microstructure
btc_returns	Bitcoin 1-bar return (cross-asset)	Cross-asset
btc_mom_5	BTC close / BTC close[–5] – 1	Cross-asset
btc_vol_ratio	BTC rv5 / BTC rv20 (vol expansion proxy)	Cross-asset
btc_trend	sign(BTC close – BTC EMA50)	Cross-asset
vol_regime	rv5 / rv20 (symbol vol expansion)	Regime
trend_strength	EMA50 slope / ATR (ADX proxy)	Regime
price_vs_sma200	(close – SMA200) / SMA200, clipped ±1	Regime
btc_corr_20	20-bar rolling Pearson corr vs BTC returns	Regime
funding_divergence	symbol_funding – btc_funding	Regime

4. Data Collection Pipeline

Training data is collected by ml/collect_data.py. Binance Futures is used as the primary klines and funding rate source (deeper history than Bybit). Bybit provides open interest and long/short ratios for recent bars. All API calls are proxied through the same HTTP proxy as production.

Source	Data	Coverage
Binance Futures	1h OHLCV klines	3 years for majors, 18 months for alts
Binance Futures	8h funding rate history	Full history paginated
BTC reference	BTC klines + funding	3 years, fetched first as cross-asset base

A data quality gate rejects any symbol where more than 50% of return bars are exactly zero — this filters delisted or illiquid assets that would corrupt quantile labels (e.g. FTM after delisting). The gate is applied immediately after kline fetch before any feature engineering.

Parameter	Value
Symbols	27 USDT perpetuals
Majors (3yr)	BTC, ETH, BNB, SOL, XRP, DOGE, AVAX, LINK, ADA, MATIC, LTC, DOT, ATOM, TRX

Alts (18mo)	AAVE, UNI, SUI, ARB, OP, APT, NEAR, INJ, TIA, SEI, WIF, 1000PEPE, BONK, LDO
Excluded	FTMUSDT — 75% zero-return bars due to delisting artefact
Total samples	187,515 after feature engineering and quality filtering
Label balance	SELL 91,250 (48.7%) / BUY 96,265 (51.3%)

5. Labelling: Return-Quantile Method

Labels are computed per-symbol using a return-quantile approach rather than fixed ATR barriers. For each bar, the 24-bar forward return is computed. The top 30th percentile of forward returns within that symbol's distribution is labelled BUY=1; the bottom 30th percentile is labelled SELL=0. The middle 40% is discarded — these bars represent ambiguous outcomes that add noise without learnable signal.

This approach has two key advantages over fixed barriers: (1) it adapts to each symbol's volatility regime automatically — a 3% move on SOL and a 3% move on BTC carry different statistical significance; (2) it guarantees a near-equal class distribution regardless of market direction, removing the need for asymmetric class weights.

Property	Value
Label horizon	24 bars (24 hours at 1h resolution)
BUY threshold	Top 30th percentile of 24h forward return for that symbol
SELL threshold	Bottom 30th percentile of 24h forward return for that symbol
Dropped	Middle 40% — ambiguous return outcomes
Result	Per-symbol balanced binary labels, adapts to volatility regime

6. Training Procedure

6.1 Per-Symbol Walk-Forward Cross-Validation

Training uses a per-symbol expanding walk-forward split with a 24-bar embargo gap between train and validation windows. The embargo prevents label leakage from the 24-bar quantile label horizon — without it, a bar near the train/val boundary in the training set would have its label computed from bars in the validation set.

For each fold and each symbol independently: the first 78% of chronological rows form the training window; the next 22% form the validation window with a 24-bar gap in between. Masks are unioned across all symbols per fold. This prevents cross-symbol contamination: BTC validation data never overlaps with ETH training data from the same calendar period.

Parameter	Value
CV folds	3 (expanding windows per symbol)
Train fraction	~78% per symbol per fold

Val fraction	~22% per symbol per fold
Embargo	24 bars between train and val (matches label horizon)
Scaler fitting	StandardScaler fitted on train split only — no val leakage
Feature clipping	±5σ after scaling

6.2 Loss Function: Cost-Sensitive Focal Loss

Training uses a custom focal loss that combines standard focal weighting (γ=2.0, down-weights easy examples) with an asymmetric cost multiplier (cost_wrong=2.0) that penalises confident wrong predictions twice as hard. This matters for trading: a high-confidence incorrect signal causes larger losses than an uncertain one.

6.3 Hyperparameters

Hyperparameter	Value
Epochs	50 (with patience=10 early stopping)
Batch size	512
Optimizer	AdamW (lr=3e-4, weight_decay=1e-4)
LR schedule	CosineAnnealingLR (T_max=50, η_min=lr×0.01)
Loss	Cost-sensitive focal loss (γ=2.0, cost_wrong=2.0)
Gradient clipping	1.0 (max norm)
Instance type	AWS SageMaker ml.g5.12xlarge (4× NVIDIA A10G GPU)

7. Backtesting & Evaluation

Evaluation uses a proper TP/SL simulation over the out-of-sample validation set (41,634 samples across 27 symbols). For each predicted signal, a position is entered at the signal bar's close price. Subsequent bars are scanned for TP or SL hits; if neither is hit within the 24-bar horizon, the trade exits at the horizon price. A round-trip fee of 0.08% (2 × 0.04% Bybit taker) is deducted from each trade.

Metric	All signals	Conf ≥ 60%	Conf ≥ 65%
Val accuracy	55.3%	—	—
Win rate (TP hit)	49.6%	49.8%	49.6%
Avg trade return	+0.649%	+0.661%	+0.654%
Profit factor	1.82	1.83	1.82
Sharpe ratio	+5.52	+5.62	+5.55
Avg win	+2.911%	+2.918%	+2.920%
Avg loss	–1.577%	–1.580%	–1.580%
Total val PnL	+27,025%	+10,448%	+5,706%

Trade count (val)	41,634	15,796	8,732

The model's positive EV derives from the asymmetric risk-reward setup rather than a high win rate. At TP=3%/SL=1.5% (2:1 R:R), break-even requires only 33.3% wins; the model achieves ~49.6%, yielding a theoretical EV of 0.496×3% – 0.504×1.5% – 0.08% = +0.65% per trade.

Confidence filtering (≥0.65) improves trade quality marginally but the win rate plateau at 49–50% across all confidence bins indicates the win rate is determined by the R:R geometry, not model confidence. The correct lever for higher absolute returns is a wider R:R ratio, not a confidence gate.

7.1 Per-Symbol Sharpe (Top 10)

Symbol	Val Acc	Sharpe	Trades
MATICUSDT	58.9%	+20.44	856
XRPUSDT	57.6%	+11.19	1,985
LTCUSDT	57.3%	+9.17	2,054
TIAUSDT	54.3%	+8.87	1,078
WIFUSDT	52.0%	+8.07	1,122
ETHUSDT	55.7%	+8.40	1,949
SOLUSDT	57.3%	+8.38	2,124
INJUSDT	51.9%	+7.40	1,090
LINKUSDT	56.2%	+5.29	2,075
BNBUSDT	54.8%	+5.16	2,059

7.2 Model Calibration

The model is well-calibrated with an Expected Calibration Error (ECE) of 0.04. Higher-confidence predictions correspond to meaningfully higher accuracy:

Confidence bin	Count	Avg conf	Accuracy	ECE gap
0.50 – 0.60	25,838	54.7%	52.7%	+0.020
0.60 – 0.70	11,304	64.2%	56.7%	+0.074
0.70 – 0.80	4,361	74.2%	66.8%	+0.075
0.80 – 0.90	131	80.5%	69.5%	+0.111

8. Inference Pipeline

The model singleton loads from ml/output/model.pt and ml/output/config.json once at application startup. For each of 20 symbols every 15 minutes:

- Fetch 200 most recent 1h candles from Bybit via the configured proxy

- Fetch BTC klines + funding once for cross-asset features
- Fetch per-symbol funding rate history (50 bars)
- Engineer all 28 features using the same computation graph as training
- Scale using saved scaler mean/scale; clip to ±5σ
- Slice the last 48 timesteps as the model input sequence
- Forward pass through CNN front-end → Transformer → head → softmax
- Apply confidence gate: skip signal if |p_buy – p_sell| < 0.10
- Append RSI/funding context to the rationale string
- Fall back to heuristic momentum scoring if model weights are absent

9. Auto-Trader

The auto-trader runs automatically after every signal refresh cycle. For each user with auto-trading enabled, it fetches live positions and wallet equity, closes positions where the signal has flipped or confidence has dropped below threshold, and opens new positions for qualifying signals within the configured risk limits. All actions are logged to the auto_trade_logs table.

Parameter	Default	Description
enabled	false	Master on/off switch
demo	true	Use Bybit Demo (api-demo.bybit.com) by default
confidence_threshold	0.65	Minimum model confidence to enter a trade
max_positions	3	Maximum concurrent auto-trade positions
position_size_pct	5%	Equity percentage per position
leverage	1×	Position leverage
tp_pct	3.0%	Take-profit distance from entry
sl_pct	1.5%	Stop-loss distance from entry (2:1 R:R)
daily_loss_limit	$50	Maximum daily loss before pausing
symbols	8 majors	BTC/ETH/SOL/BNB/DOGE/LTC/TRX/XRP

Demo mode uses Bybit's demo trading environment (api-demo.bybit.com), which executes orders against live market prices with virtual funds. Demo API keys are stored in the same database fields as testnet keys and routed via pybit's demo=True flag. This enables full end-to-end testing of the auto-trader logic without financial risk.

10. REST API Reference

10.1 Authentication

All endpoints except /api/health require Bearer JWT. Tokens are issued on POST /api/auth/register and /api/auth/login, expire after 24h. Rate limits: login 5/min, register 10/min.

10.2 Trade Endpoints

Method	Path	Description
GET	/api/trade/klines	Klines (≤1,000 candles, paginated)
GET	/api/trade/symbols	All USDT perps sorted by 24h volume
GET	/api/trade/positions	Open positions (?demo=true for demo account)
GET	/api/trade/wallet	Equity, available balance, unrealised PnL
GET	/api/trade/orders	Open orders
POST	/api/trade/order	Market order — symbol, side, qty, tp?, sl?
POST	/api/trade/close	Close position (reverse market order)
POST	/api/trade/leverage	Set symbol leverage

10.3 Auto-Trade Endpoints

Method	Path	Description
GET	/api/auto-trade/config	Get user's auto-trade configuration
POST	/api/auto-trade/config	Save auto-trade configuration
GET	/api/auto-trade/stats	Aggregate stats: fills, errors, avg conf, hold time
GET	/api/auto-trade/pnl	Closed P&L from Bybit history with summary metrics
GET	/api/auto-trade/log	Raw trade log (last 50 actions)

10.4 Signal & Account Endpoints

Method	Path	Description
GET	/api/signals	Latest signal per ticker, sorted confidence DESC
GET	/api/account/stats	Portfolio metrics, API key status
GET	/api/account/model-info	Loaded model architecture and val accuracy
POST	/api/account/api-keys	Save mainnet Bybit credentials (Fernet-encrypted)
POST	/api/account/testnet-keys	Save demo Bybit credentials (Fernet-encrypted)
POST	/api/market/refresh	Trigger manual signal + price refresh (2/min limit)

11. Bybit Integration & Demo Mode

All Bybit calls use the pybit.unified_trading.HTTP client (v5 API, linear/USDT-perp category). Per-user API keys are retrieved from PostgreSQL, decrypted with Fernet, and passed to pybit for each request. The server-level BYBIT_API_KEY environment variable provides a fallback for unauthenticated public endpoints (klines, tickers).

Mode	Endpoint	Keys used	Notes
Live	api.bybit.com	Mainnet API keys	Real money, real positions
Demo	api-demo.bybit.com	Demo API keys	Virtual funds, live prices
Public	api.bybit.com (no auth)	None	Klines, symbols, funding

Demo mode uses pybit's native demo=True parameter, which routes to Bybit's official demo trading environment. Demo keys are stored in the same bybit_testnet_key/secret fields in PostgreSQL and encrypted with Fernet.

12. Database Schema

12.1 Users

Column	Type	Notes
id	UUID	Primary key, uuid4
email	VARCHAR(320)	Unique, indexed
hashed_password	VARCHAR(128)	bcrypt
name	VARCHAR(120)	
plan	VARCHAR(40)	Default 'Private Beta'
bybit_api_key	VARCHAR(256)	Nullable, mainnet, Fernet-encrypted
bybit_api_secret	VARCHAR(256)	Nullable, mainnet, Fernet-encrypted
bybit_testnet_key	VARCHAR(256)	Nullable, demo, Fernet-encrypted
bybit_testnet_secret	VARCHAR(256)	Nullable, demo, Fernet-encrypted

12.2 Signals

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	FK → users, indexed
ticker	VARCHAR(20)	USDT perp symbol, indexed
action	VARCHAR(10)	'BUY' or 'SELL'
confidence	FLOAT	[0.0, 1.0] softmax probability
rationale	TEXT	Human-readable explanation
created_at	TIMESTAMPTZ	UTC

12.3 AutoTradeConfig

Column	Type	Notes
user_id	UUID	FK → users, unique (one config per user)

enabled	BOOLEAN	Master toggle
demo	BOOLEAN	Demo mode (default true)
confidence_threshold	FLOAT	Default 0.65
max_positions	INT	Default 3
position_size_pct	FLOAT	Default 5.0
leverage	INT	Default 1
tp_pct	FLOAT	Default 3.0
sl_pct	FLOAT	Default 1.5
daily_loss_limit	FLOAT	Default 50.0 USD
symbols	TEXT	Comma-separated, default 8 majors

13. Security Model

- Passwords hashed with bcrypt (passlib, 12 rounds)
- JWTs signed with HS256; secret rotatable via JWT_SECRET env var; 24h expiry
- Bybit API keys encrypted at rest with Fernet symmetric encryption; FERNET_KEY in .env
- Keys are decrypted in memory per-request, never returned to the client
- Per-user key isolation: each trade call resolves credentials from the authenticated user's DB row
- Input validation via Pydantic field_validators: symbol regex, side enum, qty/leverage range checks
- Rate limiting: 5/min login, 10/min register, 10/min order placement, 2/min signal refresh
- CORS restricted to configured origin list (ALLOWED_ORIGINS env var)
- Structured JSON request logging with request IDs and duration
- Global 500 handler suppresses stack traces from client responses

14. AWS SageMaker Training Pipeline

Training is triggered manually via ml/launch_sagemaker.py, which packages train.py into a tar.gz, uploads the dataset CSV to S3, and submits a CreateTrainingJob request using boto3. The training container is a pre-built PyTorch GPU image from the AWS ECR registry.

Parameter	Value
Account	125499242423
Region	us-east-1
IAM Role	astrax (EC2 instance profile)
S3 Bucket	astraiosbucket
Training image	pytorch-training:2.1.0-gpu-py310-cu121-ubuntu20.04-sagemaker
Instance	ml.g5.12xlarge (4× NVIDIA A10G, 48 GB VRAM total)

Max runtime	7,200 seconds
Output	model.pt + config.json → model.tar.gz → auto-downloaded to ml/output/

15. Frontend Architecture

The React SPA is built with Vite 6 and served by the FastAPI process via StaticFiles. Three routes: landing (/), login (/login), dashboard (/dashboard). No UI framework — pure CSS with design tokens matching the dark editorial theme (--paper:#050505, --ink:#fff, --accent:#2ecb71).

- • React 19 with hooks; no class components
- • lightweight-charts v4 — candlestick + volume panels, 1s live candle polling
- • Global Live/Demo toggle — persisted in localStorage; switches all Bybit calls
- • Mode-guarded fetch: in-flight requests from old mode are discarded on switch (modeRef pattern)
- • Trade terminal: 546-symbol searchable selector, leverage (1–100×), quick-size buttons, TP/SL
- • Auto-trade panel: Demo/Live toggle, 7 risk params, performance stats grid, realised P&L; table
- • Responsive breakpoints: 980px (tablet) and 640px (mobile)
- • Signal cards tiered by confidence with colour-coded BUY/SELL badges

16. Signal Symbol Universe

The signal engine covers 20 USDT perpetual futures across three tiers. Tier 1 symbols (strong model edge, 55–59% val accuracy) are the default auto-trade candidates. Tier 2 provides informational signals. Tier 3 symbols are below the majority-class baseline and should not be auto-traded.

Symbol	Tier	Val Acc	Sharpe	Notes
BTCUSDT	1	56.4%	+1.93	Auto-trade default
ETHUSDT	1	55.7%	+8.40	Auto-trade default
SOLUSDT	1	57.3%	+8.38	Auto-trade default
BNBUSDT	1	54.8%	+5.16	Auto-trade default
DOGEUSDT	1	56.3%	-3.14	Auto-trade default
LTCUSDT	1	57.3%	+9.17	Auto-trade default
TRXUSDT	1	56.8%	-2.80	Auto-trade default
XRPUSDT	1	57.6%	+11.19	Auto-trade default
AVAXUSDT	2	55.7%	+2.05	Informational
LINKUSDT	2	56.2%	+5.29	Informational
ADAUSDT	2	56.5%	+0.16	Informational
DOTUSDT	2	56.7%	-3.00	Informational
MATICUSDT	2	58.9%	+20.44	Informational (low sample count)

AAVEUSDT	2	52.7%	-2.27	Informational
SUIUSDT	3	52.3%	+0.39	Below baseline
ARBUSDT	3	53.4%	-1.24	Below baseline
1000PEPEUSDT	3	52.9%	+6.05	Below baseline
WIFUSDT	3	52.0%	+8.07	High vol, low acc
NEARUSDT	3	53.9%	+3.09	Below baseline
INJUSDT	3	51.9%	+7.40	Below baseline

This document describes the Astraios platform as of May 2026. Model performance metrics are out-of-sample backtests on historical data; past performance does not guarantee future results. Cryptocurrency derivatives trading involves significant risk of loss. Astraios is a technology platform — not investment advice.